CSC336 · WEB TECHNOLOGIES

Lab 4
Your First API Endpoint

A minimal Express server with one GET route and a plain HTML page that fetches and renders its JSON response.

Student: Hussnain Ali · Reference theme: Tesla

01 · SERVER ROUTE

GET /api/products

The server defines exactly one API route. It returns a hardcoded array of five products with status code 200. Express automatically serializes the array as JSON.

```
app.get("/api/products", (request, response) => {
  response.status(200).json(products);
});
```

Express server | One route only | HTTP 200

02 · RAW JSON RESPONSE

Browser API response

Opening the endpoint directly in the browser displays the raw JSON response required by the lab.


```
[
  {"id": 1, "name": "Model S", "price": 89990},
  {"id": 2, "name": "Model 3", "price": 42490},
  {"id": 3, "name": "Model X", "price": 99990},
  {"id": 4, "name": "Model Y", "price": 44990},
  {"id": 5, "name": "Cybertruck", "price": 79990}
]
```


5 products returned **JSON response** **Status 200**

03 · FRONTEND FETCH

Rendering API data

The HTML page calls the endpoint with **fetch()**. After the response is converted with **response.json()**, JavaScript creates product cards in the page without hardcoding the displayed list.

```
const response = await fetch("/api/products");
const products = await response.json();
list.innerHTML = products.map(...).join("");
```

Fetch API | Dynamic rendering | No page reload